FIAP · Global Solution 2026.2 · CitizenTech · Engenharia de Software 4ES

Caminho Livre: Deploy Blindado

Secure DevOps Tools & Cloud Computing · Prof. Gabriel Henrique Boyczuk Arcanjo

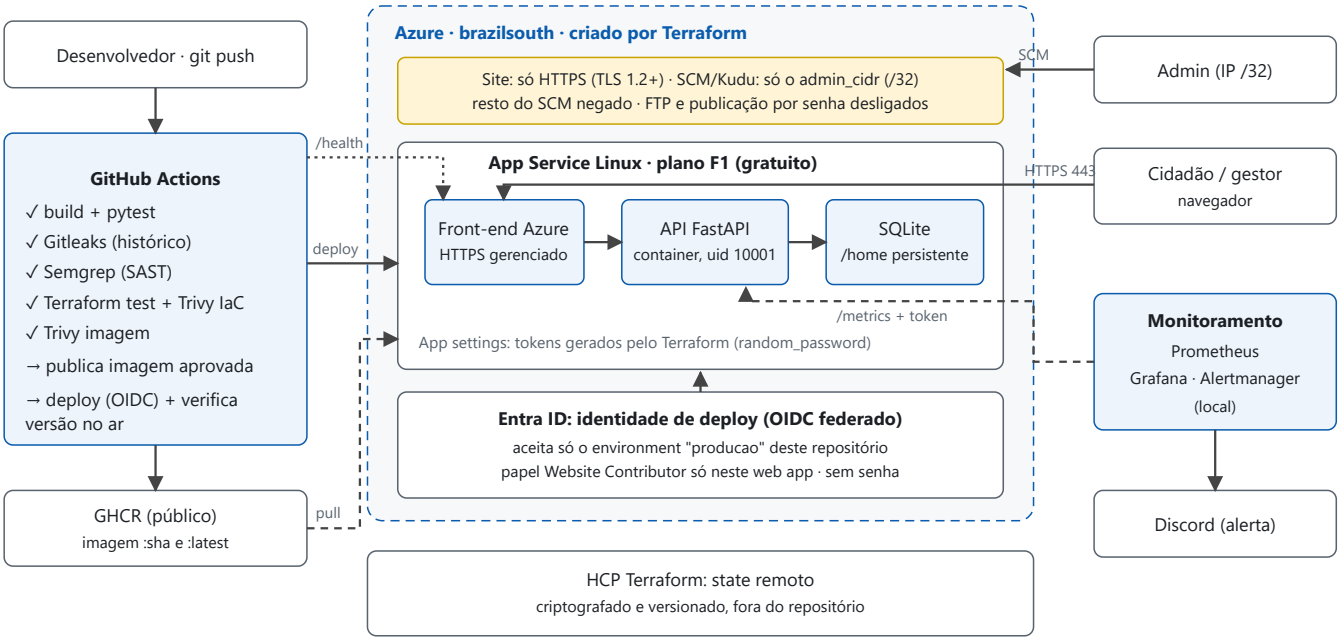
Integrante (nome completo)	RM
Guilherme Rocha Bianchini	RM97974
Nikolas Rodrigues Moura dos Santos	RM551566
Thiago Jardim de Oliveira	RM551624
Rodrigo Brasileiro	RM98952

Link	Endereço
MVP publicado (URL pública)	https://caminholivre-gs.azurewebsites.net
Repositório público	https://github.com/santos-nikolas/caminho-livre
Pipeline (GitHub Actions)	https://github.com/santos-nikolas/caminho-livre/actions

1. Ficha Técnica do MVP

Nome do produto	Caminho Livre
Cidadão e território	Pedestres com deficiência ou mobilidade reduzida, idosos e cuidadores. O piloto cobre o entorno do campus FIAP Paulista (São Paulo, SP).
Problema, em uma frase	Rampas bloqueadas, calçadas obstruídas e travessias com barreira são relatadas de forma dispersa e ninguém acompanha se foram resolvidas.
Proposta de valor	O cidadão registra a barreira no mapa e acompanha o status. O gestor do local recebe uma fila auditável para revisar e encerrar cada relato.
Arquitetura: componentes	Página web (HTML + Leaflet) e API FastAPI num container, publicado no Azure App Service (Linux, plano F1) criado por Terraform. Banco SQLite no armazenamento persistente do App Service. CI/CD com GitHub Actions e deploy por OIDC. Prometheus, Grafana e Alertmanager.
Fontes de dados	OpenStreetMap (mapa base e malha viária) e relatos do próprio MVP. Os pontos de demonstração são sintéticos e vêm identificados na tela.
Modelos de IA utilizados	Detector de obstáculos customizado (YOLO), do componente Applied Computer Vision. Fica fora do escopo desta entrega.
URL do MVP e repositório	Ver tabela de links acima.
Indicadores de impacto	Relatos recebidos e revisados, tempo até a primeira revisão e taxa de resolução confirmada. São indicadores de processo, não prova de melhoria física.
ODS conectados	ODS 10 (redução das desigualdades) e ODS 11 (cidades e comunidades sustentáveis).

2. Arquitetura



Fluxo: cada `git push` passa por 5 portões (testes, Gitleaks, Semgrep, Trivy IaC e Trivy imagem). Só a imagem aprovada na `main` é publicada no GHCR (tags `:sha` e `:latest`). O job `deploy` faz login na Azure por OIDC, com um token de curta duração e sem senha guardada. Em seguida, aponta o App Service para a imagem `:sha` do commit e só passa quando `/health` responde com esse SHA.

Por que PaaS (App Service) e não VM: a assinatura disponível para a equipe é a *Azure para Estudantes Starter*. Ela só permite os provedores `Microsoft.Web`, `Microsoft.Sql` e alguns de gestão; VM, rede virtual e Storage Account são negados pela própria assinatura (`DisallowedProvider`). O plano F1 do App Service é gratuito, roda o mesmo container do `docker compose` local e entrega HTTPS e patches do host pela plataforma. Em troca, ele tem limites: 60 min de CPU por dia, sem *Always On* e uma instância só (seção 10).

3. Container e Infraestrutura como Código (5,0)

3.1 Dockerfile enxuto, não-root, sem segredo e stack em docker compose

Requisito	Como foi atendido	Arquivo
Enxuto	Multi-stage: o `pip` roda no estágio de build e só os pacotes instalados vão para a imagem `python:3.12-slim`. O banco usa o `sqlite3` da biblioteca padrão, sem driver extra. `.dockerignore` exclui testes e cache.	`app/Dockerfile`
Usuário não-root	`USER 10001`. No compose: `read_only`, `cap_drop: ALL`, `no-new-privileges` e `mem_limit 256m`. O único ponto gravável é o volume do banco.	`docker-compose.yml`
Sem segredo na imagem	Os segredos só entram por variável de ambiente em runtime: `.env` local e *app settings* na Azure. O Trivy roda com `--scanners secret` na imagem a cada push.	workflow
Patches do SO na imagem	`apt-get upgrade` no estágio final. Foi uma correção exigida pelo Trivy (seção 4.3).	`app/Dockerfile`
Stack com docker compose	`api` (com o volume `dados`) e `proxy` (Caddy, HTTPS automático quando há domínio). O override local acrescenta o `build`. Em produção, a mesma imagem roda no App Service.	`docker-compose.yml`

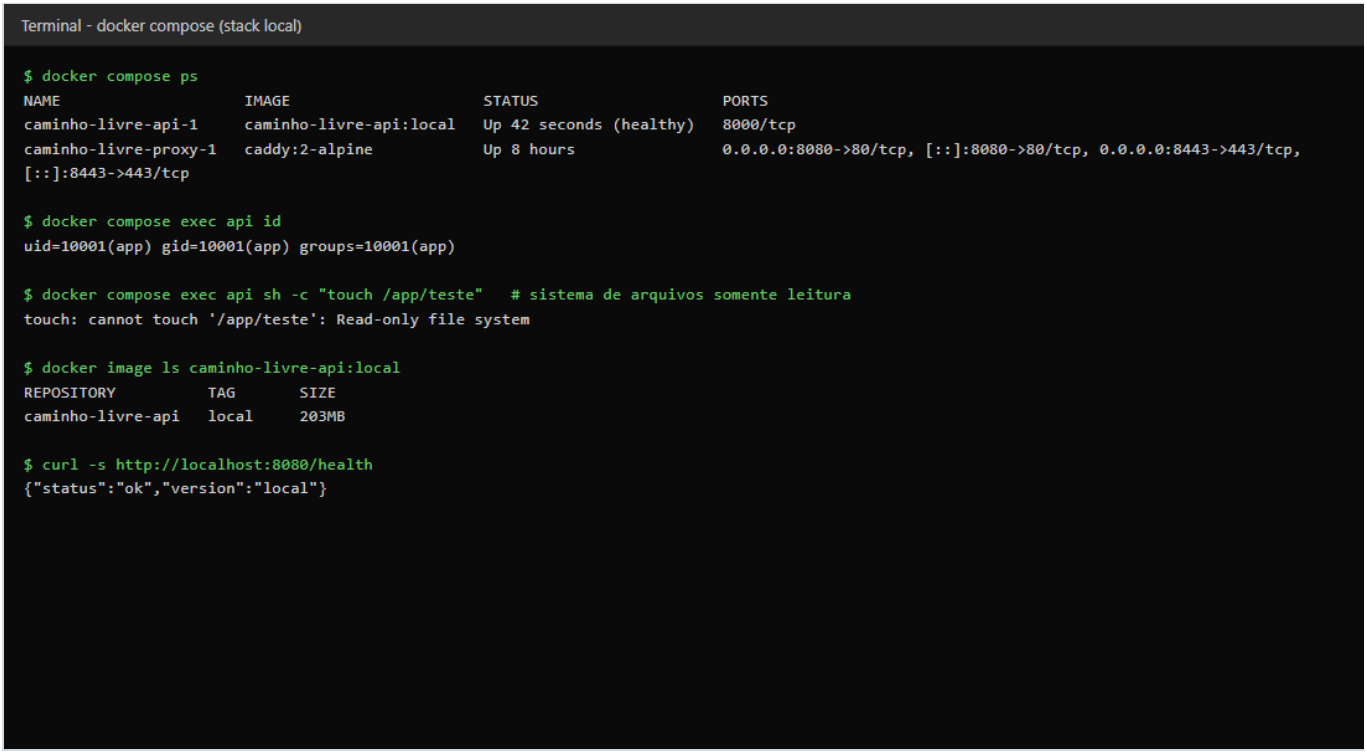


Figura 1: `docker compose ps` com a stack no ar e `docker compose exec api id` mostrando uid 10001 (não-root). A tentativa de gravar em `/app` falha porque o sistema de arquivos do container é somente leitura.

3.2 Terraform: state e .tfvars fora do repositório, porta admin nunca em 0.0.0.0/0

Requisito	Como foi atendido
Nuvem criada por código	`infra/terraform/main.tf` cria: resource group, plano App Service Linux F1 e web app de container (GHCR). Também cria a identidade de deploy no Entra ID (aplicação, *service principal* e credencial federada OIDC) e a atribuição de papel com menor privilégio.
State fora do repositório	Bloco `cloud {}`: o state fica no HCP Terraform, criptografado e versionado. A organização e o workspace vêm de variáveis de ambiente. O `*.tfstate` e o `*.tfvars` estão no `.gitignore`; o repositório só tem o `terraform.tfvars.example`.
Porta admin nunca 0.0.0.0/0	No PaaS, a porta administrativa é o site SCM/Kudu (console, logs e deploy). A regra `scm_ip_restriction` aceita só o `var.admin_cidr`, e a ação padrão do SCM é `Deny`. A *validation* só aceita prefixos de /24 a /32, então `0.0.0.0/0` é recusado no `plan`. Não há SSH, e FTP e publicação por senha estão desligados.
Sem senha digitada	Token de admin e token de métricas vêm de `random_password`. O deploy usa OIDC: o GitHub não guarda senha nem chave da Azure, só IDs.

```
$ terraform test
  run "bloqueia_admin_aberto_para_a_internet"... pass
  run "bloqueia_faixa_larga_no_admin"... pass
  run "infra_segura_com_ip_do_admin"... pass
Success! 3 passed, 0 failed.
```

O terceiro teste confere, com providers simulados:

- o SCM só aceita o IP do admin e nega o resto;
- o site exige HTTPS com TLS 1.2+ e não tem FTP;
- a publicação por senha está desligada;
- o OIDC só aceita o environment `producao` deste repositório;
- o papel de deploy vale só para este web app.

PowerShell - terraform apply + state no HCP Terraform

```
PS C:\...\caminho-livre\infra\terraform> terraform apply -input=false plano.tfplan
azurerm_linux_web_app.app: Modifying... [id=/subscriptions/cb5637fc-fe13-4cb6-bc64-3e9f655aa9d8/resourceGroups/rg-caminholivre/providers/Microsoft.Web/sites/caminholivre-gs]
azurerm_linux_web_app.app: Still modifying... [id=/subscriptions/cb5637fc-fe13-4cb6-bc64-...rs/Microsoft.Web/sites/caminholivre-gs, 00m10s elapsed]
azurerm_linux_web_app.app: Still modifying... [id=/subscriptions/cb5637fc-fe13-4cb6-bc64-...rs/Microsoft.Web/sites/caminholivre-gs, 00m20s elapsed]
azurerm_linux_web_app.app: Modifications complete after 30s [id=/subscriptions/cb5637fc-fe13-4cb6-bc64-3e9f655aa9d8/resourceGroups/rg-caminholivre/providers/Microsoft.Web/sites/caminholivre-gs]
azurerm_role_assignment.deploy: Creating...
azurerm_role_assignment.deploy: Still creating... [00m10s elapsed]
azurerm_role_assignment.deploy: Still creating... [00m20s elapsed]
azurerm_role_assignment.deploy: Creation complete after 24s [id=/subscriptions/cb5637fc-fe13-4cb6-bc64-3e9f655aa9d8/resourceGroups/rg-caminholivre/providers/Microsoft.Web/sites/caminholivre-gs/providers/Microsoft.Authorization/roleAssignments/320ff100-6f1d-3065-e1f8-709cf2ec3235]

Apply complete! Resources: 1 added, 1 changed, 0 destroyed.

Outputs:

admin_token = <sensitive>
app_name = "caminholivre-gs"
app_url = "https://caminholivre-gs.azurewebsites.net"
azure_client_id = "96b7116b-5f4d-412d-b6d0-296dbd67410f"
azure_tenant_id = "11dbbfe2-89b8-4549-be10-cec364e59551"
metrics_token = <sensitive>
resource_group = "rg-caminholivre"

$ terraform state list      # state remoto no HCP Terraform (org fiap-nikolas, workspace caminho-livre)
data.azuread_client_config.atual
azuread_application.deploy
azuread_application_federated_identity_credential.github
azuread_service_principal.deploy
azurerm_linux_web_app.app
azurerm_resource_group.rg
azurerm_role_assignment.deploy
azurerm_service_plan.plano
random_password.admin_token
random_password.metrics_token

$ terraform plan -detailed-exitcode   # infraestrutura sem desvio do codigo
No changes. Your infrastructure matches the configuration.
exit code: 0
```

Figura 2: último `terraform apply` concluído, com os outputs (os tokens aparecem como `<sensitive>`). Abaixo, os 10 recursos no state remoto e um `plan` sem mudanças, que mostra a nuvem igual ao código.

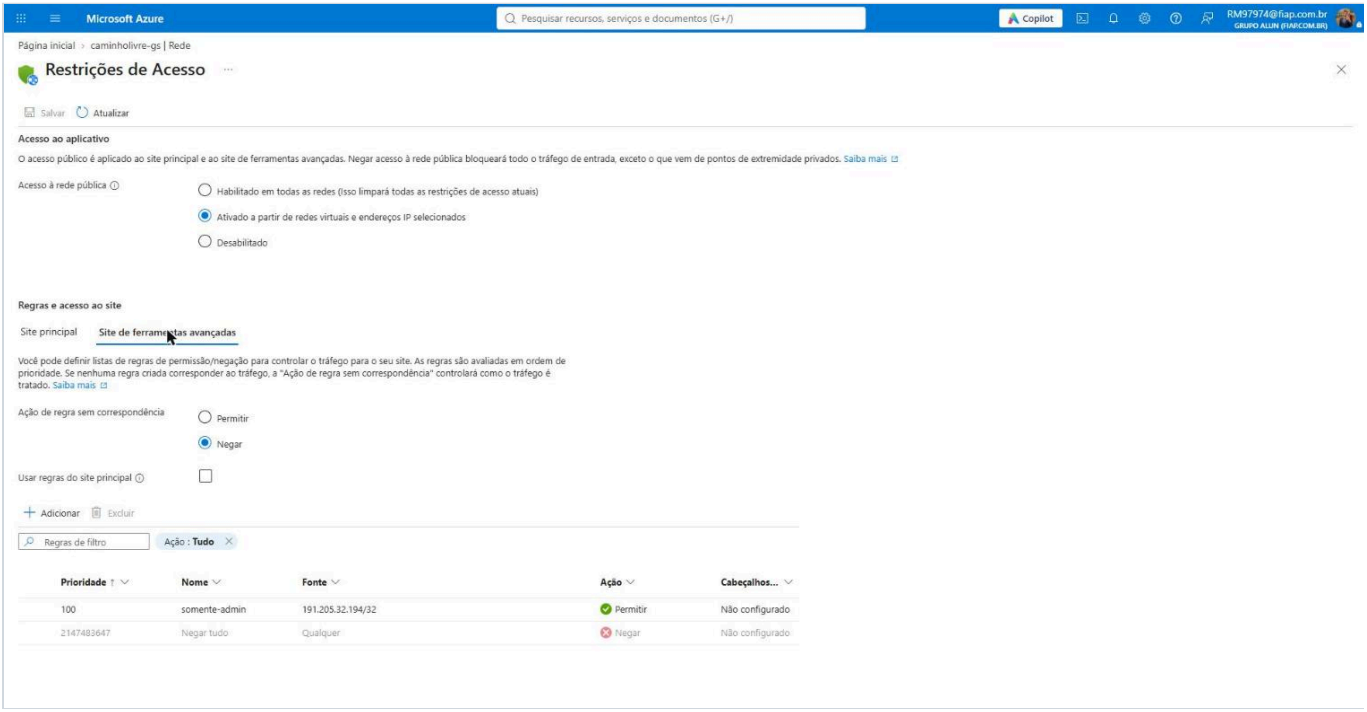


Figura 3: restrições de acesso do web app no portal Azure. O site SCM/Kudu só aceita o IP /32 do administrador e nega o resto.

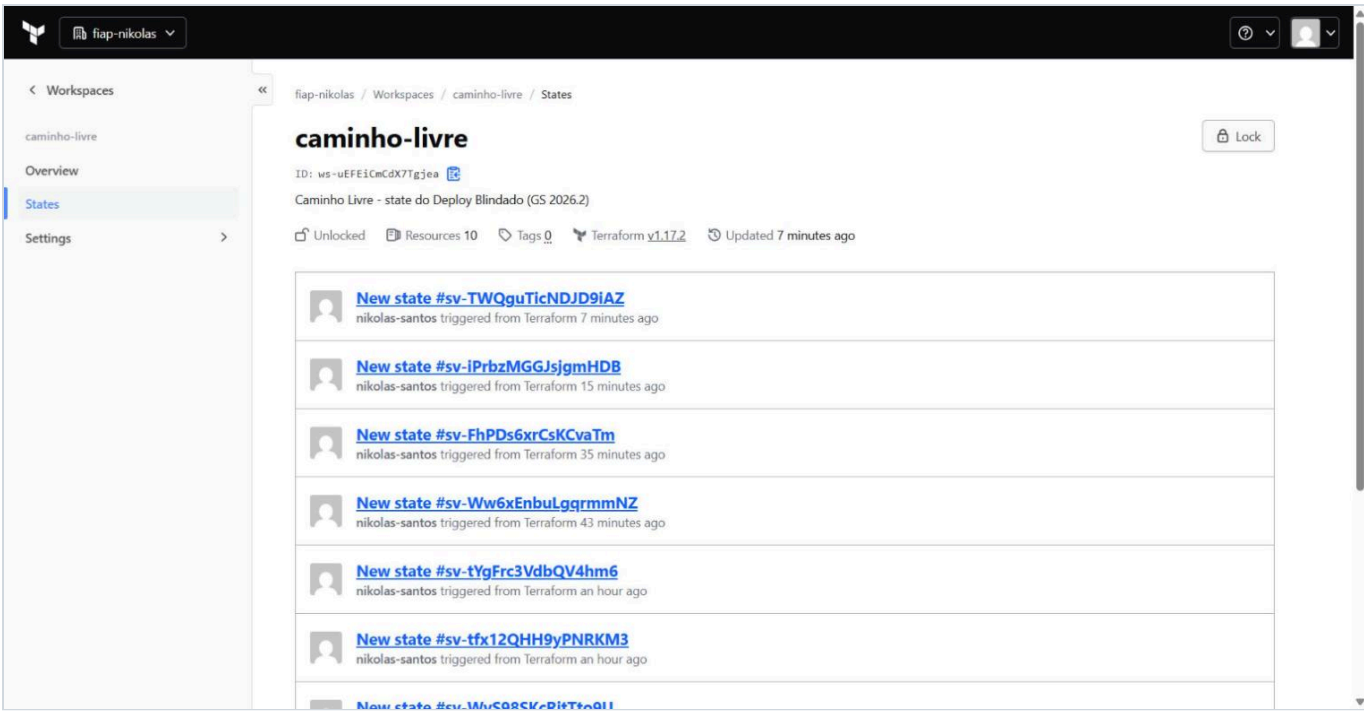


Figura 4: workspace no HCP Terraform com o state e o histórico de versões (fora do Git).

3.3 Problemas reais no provisionamento

Problema	Causa	Solução
`plan` falhava ao registrar provedores	A assinatura Starter não permite registrar provedores de recurso.	`resource_provider_registrations = "none"`. O único provedor usado, `Microsoft.Web`, já vem registrado.
403 ao criar o *service principal* e a credencial OIDC	O tenant da FIAP não torna dono quem cria a aplicação, e uma aplicação sem dono não pode ser alterada.	`owners` explícito com o usuário que roda o Terraform, lido de `azuread_client_config`.
Primeiro deploy recusado no login (`AADSTS700213`)	O GitHub passou a emitir o *subject* OIDC com os IDs imutáveis do dono e do repositório (`santos-nikolas@126473396/caminho-livre@1454716396`).	Credencial federada atualizada para esse formato. Ganho de segurança: um repositório recriado ou renomeado com o mesmo nome não herda o acesso de deploy.
Conexões com a Azure cortadas no `apply`	A rota IPv6 da rede da equipe até a Azure derrubava as conexões: 9 de 12 testes falharam por IPv6, 0 de 12 por IPv4.	O `apply` rodou por um proxy local que só sai por IPv4. Cada nova tentativa do Terraform criava só o que faltava, sem duplicar recursos.

4. Pipeline DevSecOps (4,0)

Arquivo: `.github/workflows/devsecops.yml`. Roda em **todo push**, em qualquer branch.

Job	Ferramenta	Bloqueia quando
Build e testes	pytest (API completa sobre SQLite temporário)	qualquer teste falha (fluxo cidadão/gestor, validação de território, 401 sem token, 404, LGPD)
Gitleaks	Gitleaks 8.28 (histórico completo)	algum segredo aparece em qualquer commit
Semgrep	p/default, p/python, p/dockerfile, p/secrets	algum achado aparece (`--error`)
Terraform + Trivy IaC	fmt, validate, test + `trivy config`	formatação, validação, teste de segurança ou misconfiguração HIGH/CRITICAL
Imagem + Trivy	`trivy image --scanners vuln,secret`	há CVE HIGH/CRITICAL com correção ou segredo na imagem
Deploy automático	`azure/login` (OIDC) + `az webapp config container set`, environment `producao`	o SHA não entra no ar em 10 min, ou `/metrics` responde sem token

Segredos no GitHub Secrets:

- `AZURE_CLIENT_ID`, `AZURE_TENANT_ID` e `AZURE_SUBSCRIPTION_ID`: só identificadores; quem autentica é o token OIDC de curta duração.
- `METRICS_TOKEN`: usado no smoke test autenticado.

O push para o GHCR usa o `GITHUB_TOKEN` efêmero, com `packages: write` só no job de imagem. O `id-token: write` existe só no job de deploy, e o padrão do workflow é `contents: read`.

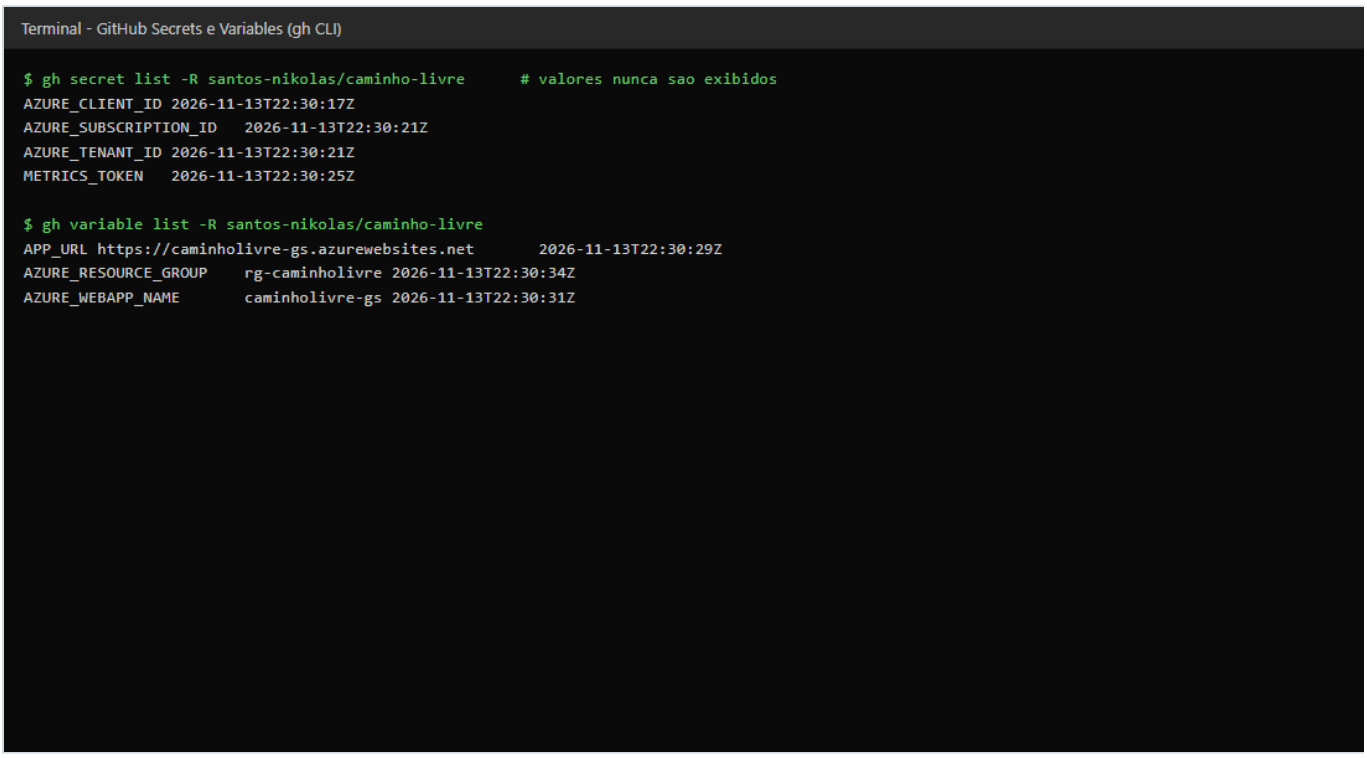


Figura 5: segredos e variáveis do repositório listados com o `gh` CLI. Os valores nunca são exibidos, só nomes e datas.

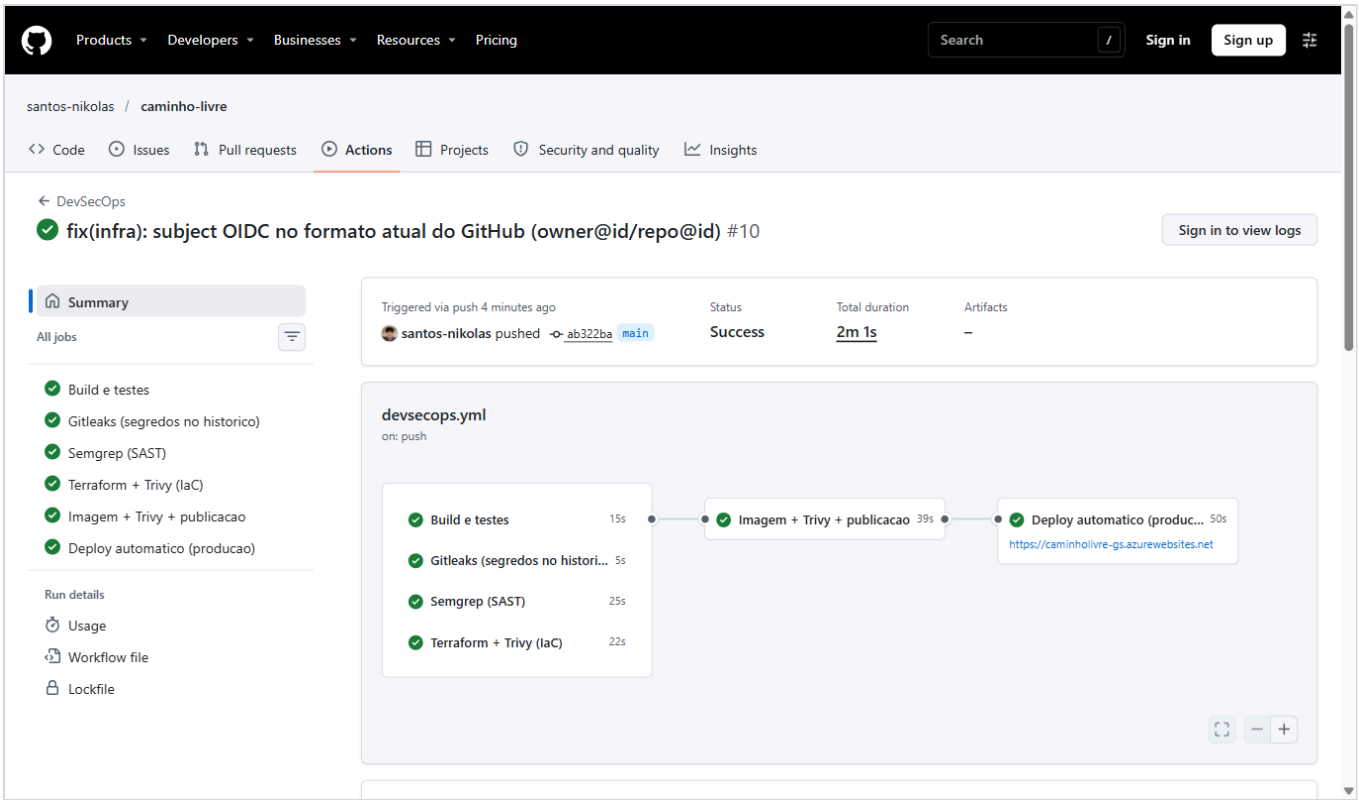


Figura 6: execução completa na `main` com todos os jobs verdes, incluindo o deploy.

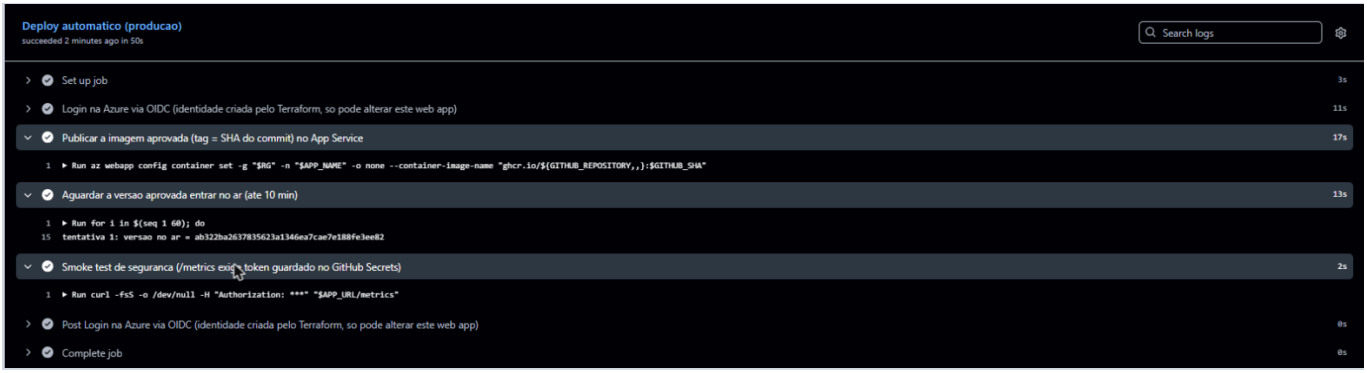


Figura 7: log do job *Deploy automático*. O App Service recebe a imagem `:SHA`, o `/health` confirma a versão `ab322ba...` no ar e o smoke test usa o token mascarado (`***`).

4.3 Execução barrada e corrigida

Caso real no desenvolvimento: na primeira varredura da imagem, o Trivy barrou o build com 2 vulnerabilidades HIGH no pacote `zlib1g` da imagem base Debian 13 (CVE-2026-85091, *heap buffer overflow*), já corrigidas na versão `1:1.3.dfsg+really1.3.1-1+deb13u1`. A correção foi aplicar os patches do SO no Dockerfile (`apt-get upgrade`). Com isso a varredura passou a mostrar 0 HIGH/CRITICAL.

Demonstração no GitHub Actions: numa branch descartável foi commitado um segredo falso (`DATABASE_PASSWORD = "..."`, gerado na hora), e o Gitleaks barrou o pipeline (execução barrada). Sem esse portão, a imagem não foi construída nem publicada. A correção foi remover o segredo *do histórico* com `git reset` + `push --force`, e não só apagar o arquivo, porque o Gitleaks varre todos os commits (execução corrigida). Num caso real, o segredo também seria rotacionado.

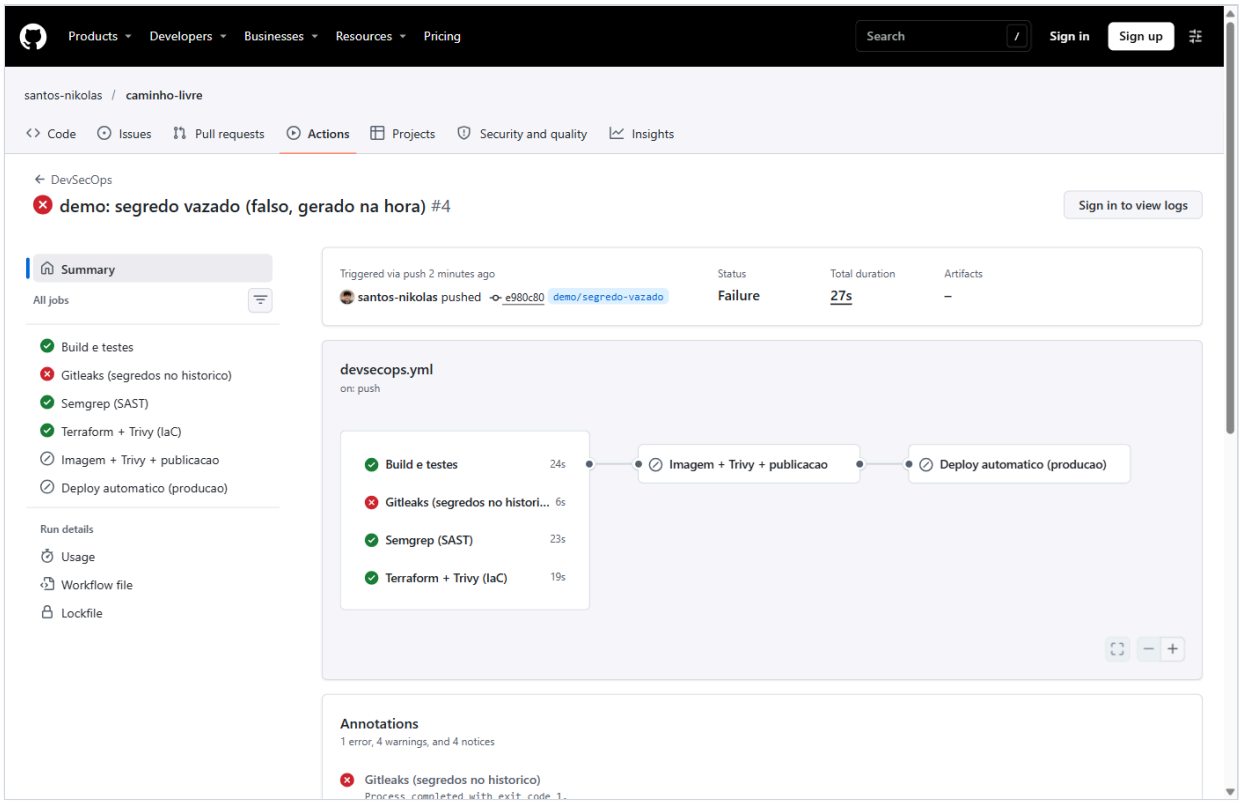


Figura 8: execução barrada pelo Gitleaks (segredo falso detectado). A imagem e o deploy não rodaram.

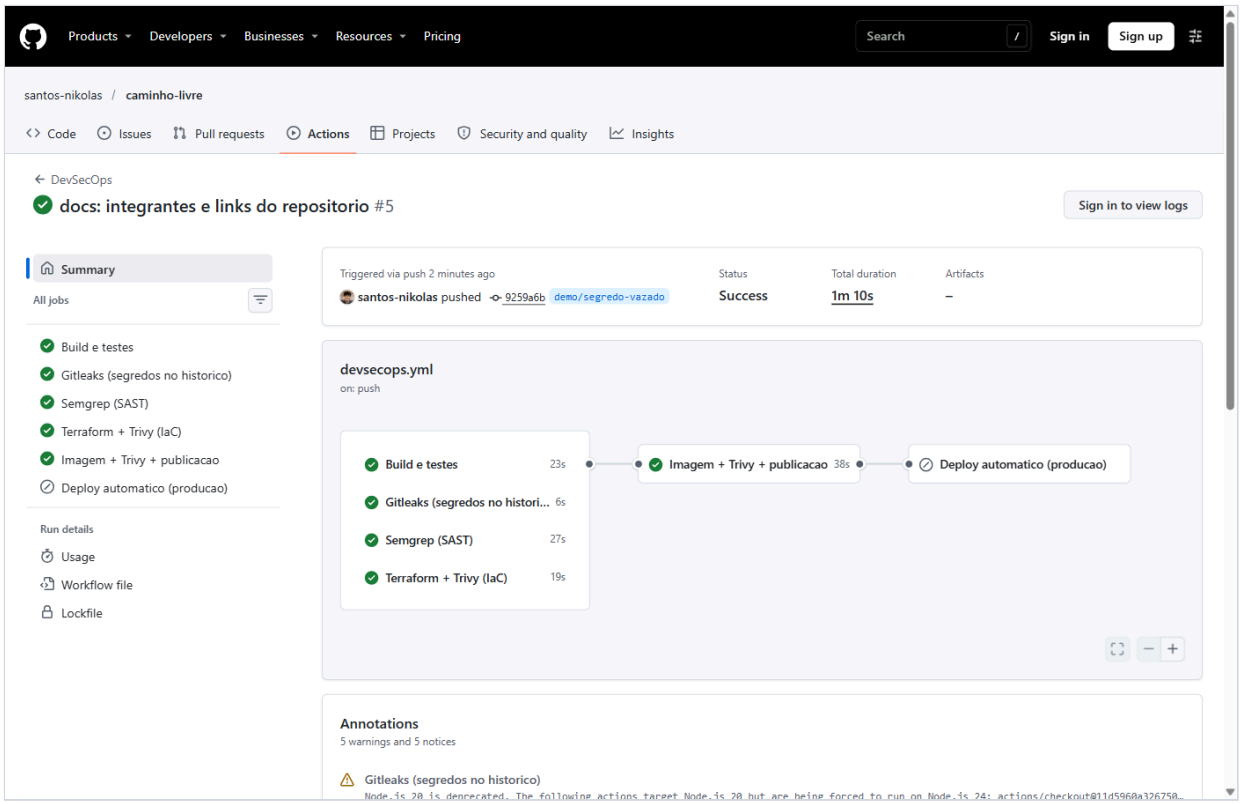


Figura 9: mesma branch, execução corrigida com todos os portões verdes.

5. Observabilidade (3,0)

A API expõe `/metrics` (Prometheus) protegido por token Bearer. Prometheus, Grafana e Alertmanager rodam localmente e raspam o MVP publicado via HTTPS, como o enunciado permite. Os rótulos de rota usam o *template* (`/api/admin/reports/{relato_id}`), o que evita cardinalidade alta e IDs nas métricas.

Sinal de ouro	**Métrica / PromQL**
Latência	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))` (p50/p95/p99)
Tráfego	`sum by (route) (rate(http_requests_total[1m]))`
Erros	`(sum(rate(http_requests_total{status=~"5.."}[5m])) or vector(0)) / sum(rate(http_requests_total[5m]))` (mostra 0% quando não há erro)
Saturação	`process_resident_memory_bytes` , `rate(process_cpu_seconds_total[1m])` (o F1 tem cota de CPU), `http_requests_in_progress`

Alerta	**Condição**	**Severidade**
CaminhoLivreFora	`up == 0` por 1 min	critical
TaxaDeErrosAlta	> 5% de respostas 5xx por 1 min	critical
LatenciaP95Alta	p95 > 1 s por 5 min	warning
SaturacaoMemoria	> 200 MB por 5 min	warning

Disparo do alerta com uma falha real (13/11/2026, horário de Brasília):

- **19:47:20:** o web app de produção foi parado com `az webapp stop` .
- **19:48:07:** a raspagem do Prometheus passou a falhar.
- **19:49:08:** depois de 1 min, o `CaminhoLivreFora` foi para *firing* e chegou ao Alertmanager.
- **19:49:39:** o app foi religado com `az webapp start` .
- **19:50:19:** o MVP voltou.
- **19:50:24:** o alerta foi resolvido.

O Alertmanager entrega no canal do Discord cujo webhook estiver em `monitoring/secrets/discord_webhook` , fora do Git. Não existe nenhum endpoint de "erro proposital" exposto em produção.

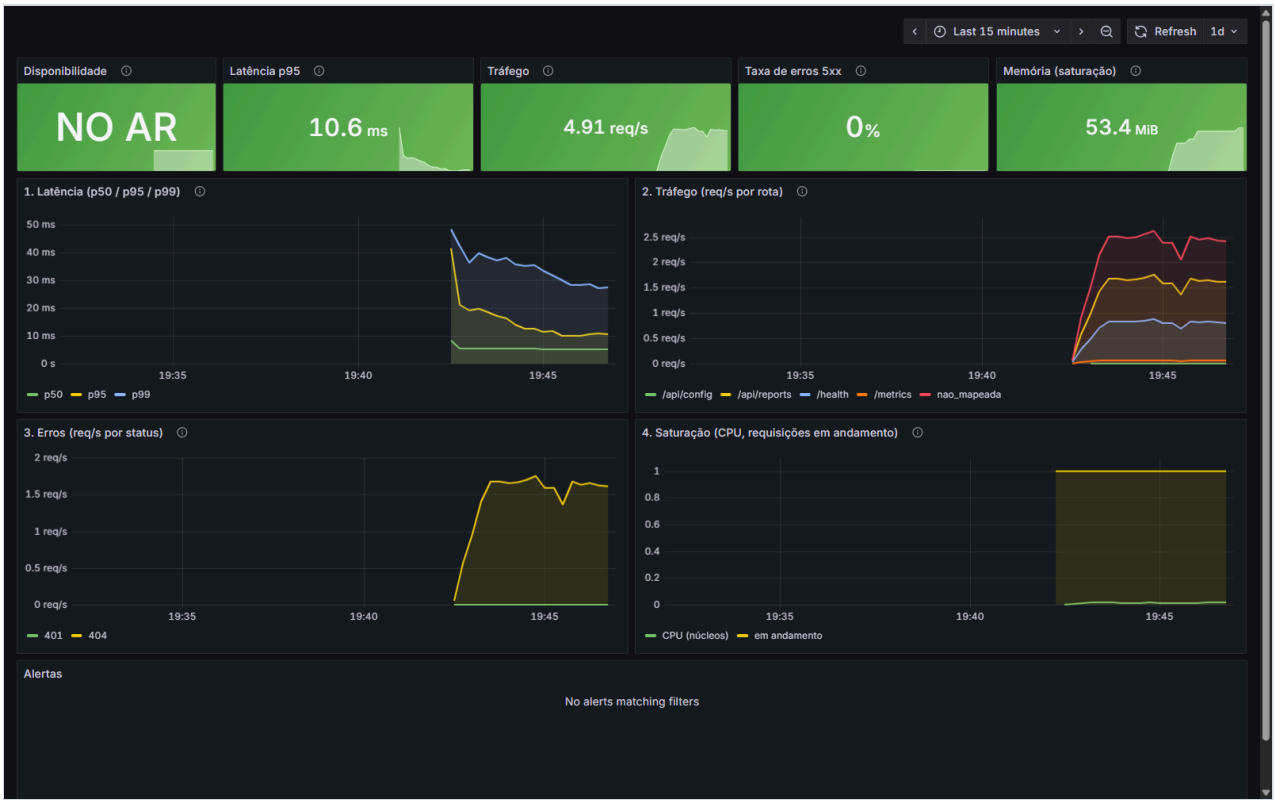


Figura 10: dashboard *Caminho Livre - 4 sinais de ouro* no Grafana, alimentado pelo MVP publicado. O tráfego vem de uma carga leve gerada pela equipe; os 404 são de uma rota inexistente incluída de propósito na carga.

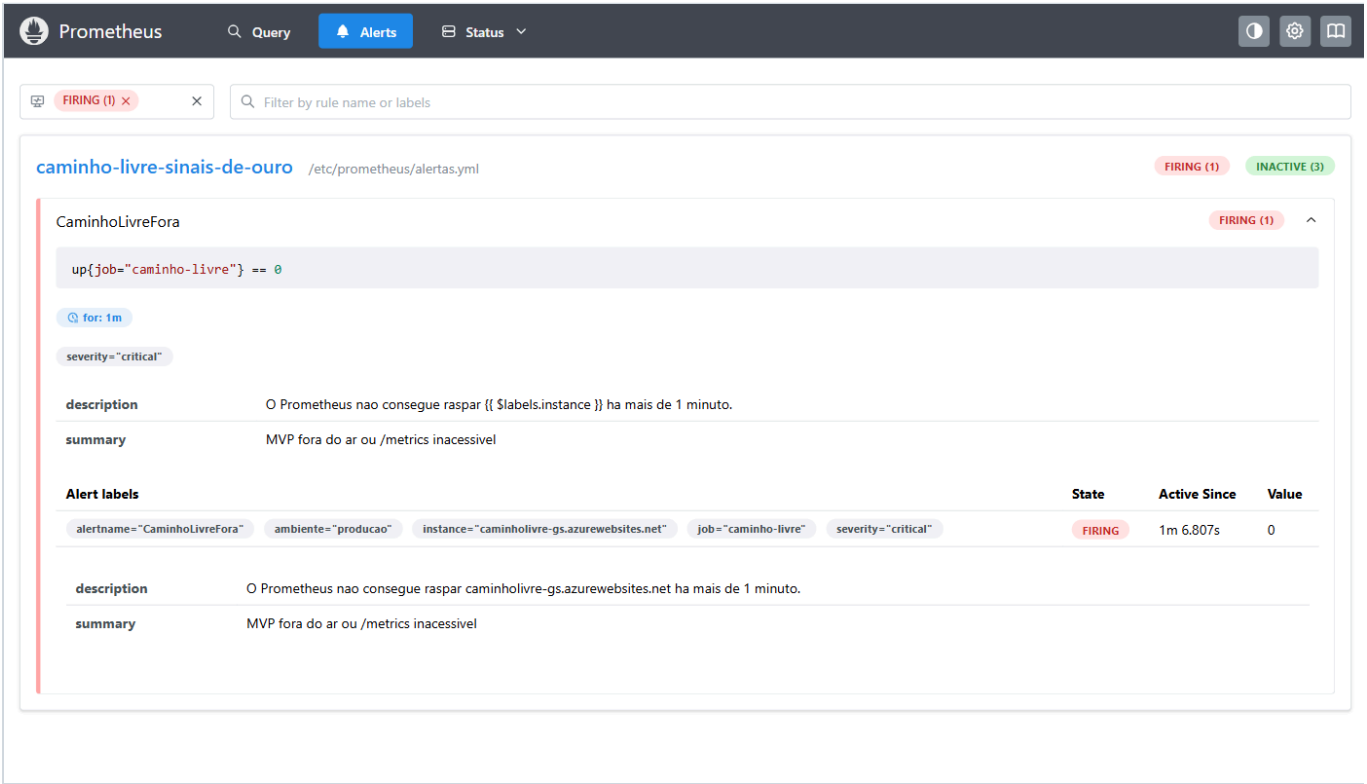


Figura 11: `CaminhoLivreFora` em *firing* no Prometheus durante a queda provocada, com a instância `caminholivre-gs.azurewebsites.net` e `up = 0`.

6. MVP publicado

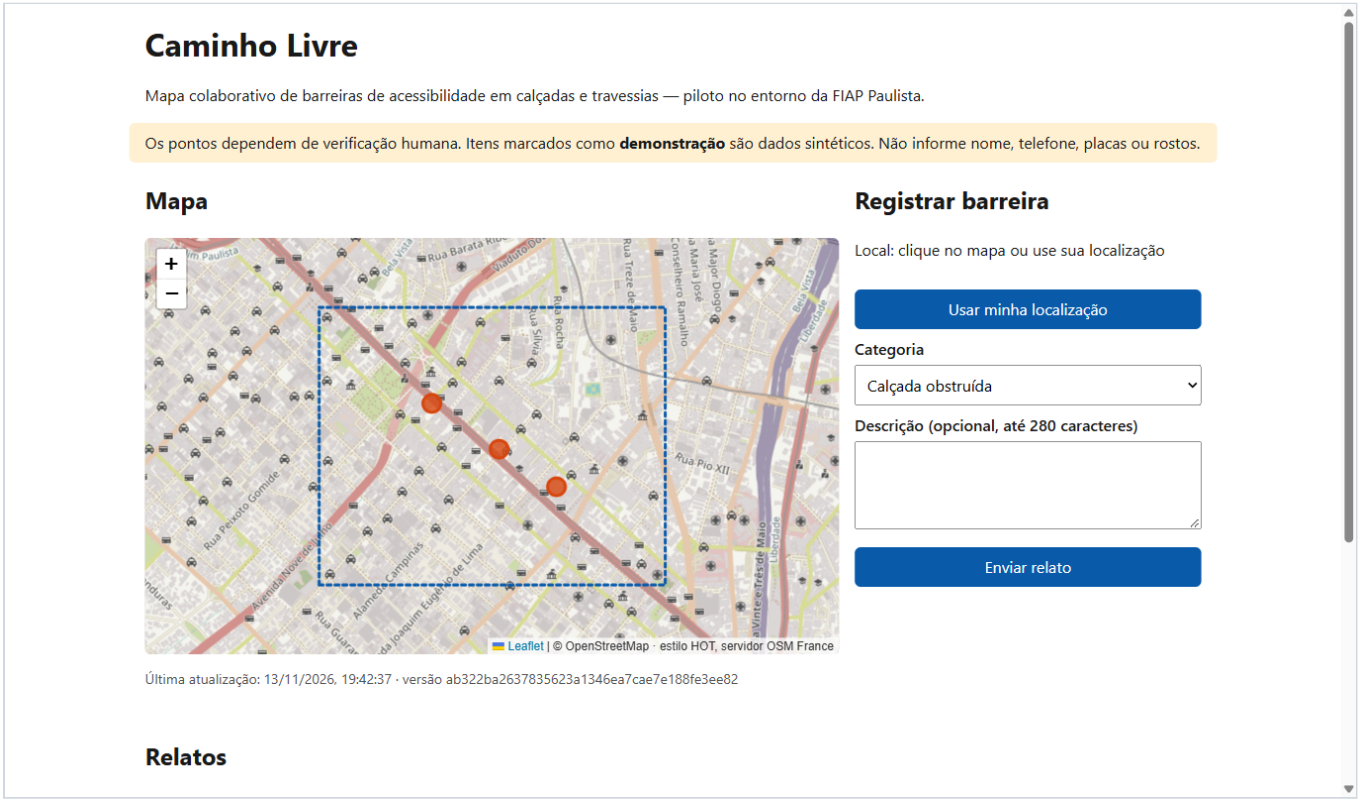


Figura 12: MVP aberto na URL pública (HTTPS), com mapa, formulário e lista de relatos.

7. Responsabilidade compartilhada (modelo PaaS)

Camada	Microsoft Azure	Equipe Caminho Livre
Física e virtualização	Datacenter, hardware, rede física, hipervisor, isolamento entre clientes	—
Rede e borda	Front-end do App Service, certificado TLS do `*.azurewebsites.net`, proteção básica contra DDoS	HTTPS obrigatório, TLS 1.2+, SCM só do IP do admin, FTP e publicação por senha desligados (Terraform)
SO e runtime do host	Patches do SO dos workers e do Docker do App Service	—
Container e dependências	—	Imagem base atualizada, CVEs (Trivy), usuário não-root, dependências fixadas
Aplicação	—	Código (testes, Semgrep), autenticação do gestor, CSP, validação de entrada
Identidade e segredos	Entra ID, emissão dos tokens OIDC, RBAC	Identidade de deploy com menor privilégio, tokens gerados pelo Terraform, GitHub Secrets, rotação
Armazenamento	Durabilidade e criptografia em repouso do `/home` do App Service	Backup do arquivo SQLite (pendente, ver riscos)
Dados e LGPD	Região escolhida (brazilsouth) e certificações da plataforma	Minimização, sanitização, retenção e resposta a titulares (controladora)
Monitoramento	Saúde da plataforma	Métricas da aplicação, alertas e resposta a incidente

8. LGPD: onde ficam os dados pessoais

Dado	É pessoal?	Onde fica	Proteção
Conta de cidadão	—	Não existe	Minimização: o piloto não tem cadastro
Coordenada do relato	Pode indicar onde a pessoa estava	SQLite em `/home/data/caminholivre.db`, armazenamento do App Service em brazilsouth	Arredondada para 4 casas (~11 m) antes de gravar
Descrição livre	Pode conter contatos	Mesmo arquivo (`descricao_sanitizada`)	E-mail, telefone e CPF trocados por `[removido]` antes de gravar (testado)
Endereço IP do visitante	Sim	Logs HTTP do App Service (brazilsouth)	Retenção de 7 dias / 35 MB, só para diagnóstico; a API não grava IP nem corpo de requisição e as métricas não têm IP
Fotos	Podem ter rostos/placas	Não são armazenadas neste MVP	O módulo de Visão processa sob demanda (fora desta entrega)
Segredos (tokens)	Não pessoais, mas sensíveis	*App settings* do App Service, state no HCP Terraform, GitHub Secrets	Nunca no Git (Gitleaks); acesso restrito por RBAC e por conta

Em trânsito, tudo passa por HTTPS (TLS 1.2+, certificado gerenciado pela Azure). Em repouso, o armazenamento do App Service é criptografado pela plataforma. Os dados pessoais ficam no Brasil, na região `brazilsouth`. O state do Terraform (no HCP Terraform) guarda os tokens da aplicação, mas nenhum dado de cidadão.

9. Estimativa de custo

Fonte: Azure Retail Prices API (preços públicos pay-as-you-go em USD, região `brazilsouth`), consultada em 13/11/2026. O mês considerado tem 730 h.

Recurso	Preço unitário	USD/mês
App Service Plan F1 (Linux)	US$ 0,00/h	0,00
Armazenamento do App Service (1 GB incluso no F1)	incluso	0,00
Entra ID (aplicação de deploy)	incluso	0,00
HCP Terraform (plano Free)	—	0,00
GitHub Actions e GHCR (repositório público)	—	0,00
Prometheus/Grafana/Alertmanager (local)	—	0,00
Total do piloto		**0,00**
Próximo degrau: App Service Plan B1 (Linux), com Always On e sem cota diária de CPU	US$ 0,02/h	≈ 14,60

O piloto cabe inteiro em planos gratuitos. Para sair do piloto, o próximo passo é o B1 (≈ US$ 14,60/mês) e, com mais de uma instância, um banco gerenciado no lugar do SQLite. Os preços do banco gerenciado não foram orçados aqui, porque o serviço não está disponível na assinatura Starter.

10. Riscos conhecidos e próximos passos

- **SQLite em armazenamento de rede:** funciona com uma instância e pouca escrita, que é o caso do piloto. Para escalar, migrar para um banco gerenciado (Azure SQL ou PostgreSQL Flexible Server).
- **Backup do banco:** o F1 não oferece backup do App Service. Próximo passo: cópia periódica do arquivo SQLite com retenção, ou um banco gerenciado com backup automático.
- **Limites do F1:** 60 min de CPU por dia e sem *Always On*. A primeira requisição depois de um período ocioso é lenta (partida a frio), e se a cota acabar o site para até o dia seguinte. O alerta `CaminhoLivreFora` cobre esse caso.
- **Monitoramento local:** só alerta enquanto a máquina da equipe está ligada. Próximo passo: Application Insights / Azure Monitor ou Grafana Cloud.
- **Rate limiting:** o POST de relatos ainda não tem limite por IP. Próximo passo: limite na aplicação ou um gateway na frente do App Service.
- **Segredos em app settings:** a assinatura Starter não oferece Key Vault. Com uma assinatura completa, os tokens iriam para o Key Vault, lidos por *managed identity*.

11. Checklist da rubrica

Critério	Pts	Evidência
Dockerfile enxuto, não-root, sem segredo e docker compose	2,0	Seção 3.1, Fig. 1, `app/Dockerfile`, `docker-compose.yml`
Terraform, state/.tfvars fora do repo, admin nunca 0.0.0.0/0	3,0	Seção 3.2, `terraform test`, Figs. 2–4, `.gitignore`
Actions: build, Gitleaks, Semgrep, Trivy por push; Secrets; deploy automático; barrada e corrigida	4,0	Seção 4, Figs. 5–9
Dashboard com os 4 sinais de ouro e um alerta disparado	3,0	Seção 5, Figs. 10–11
Arquitetura, responsabilidade compartilhada, LGPD, custo, README	2,0	Seções 2, 7, 8 e 9; README com Ficha Técnica

12. Conclusão

O Caminho Livre chega ao cidadão por um caminho em que nenhum passo é manual. A infraestrutura é descrita em Terraform e testada antes de existir. Cada alteração de código passa por testes e por quatro varreduras de segurança, e só a imagem aprovada é publicada e implantada, por uma identidade sem senha e com permissão mínima. Depois de publicado, o produto é observado pelos quatro sinais de ouro, com um alerta disparado por uma falha real. O custo do piloto é zero. Os riscos que ainda existem, principalmente backup e escala do banco, estão listados com o próximo passo de cada um.